# Lab 11 — Antwoorden met toelichting

*Elk antwoord volgt hetzelfde schema: het antwoord, het mechanisme, de nuance of valkuil, en een voorbeeld dat je aan de terminal kunt nagaan.*

---

### Vraag 1 — Twee Compose-VM's tegenover een cluster

**Antwoord.** (1) **Een VM valt uit**: haar containers zijn dood tot iemand ingrijpt; in een cluster stelt de *controller manager* vast dat er Pods ontbreken en plaatst de *scheduler* ze opnieuw op een gezonde node. (2) **Opschalen**: van twee naar zes instanties vraagt vier VM's om klaar te zetten; in een cluster volstaat `replicas: 6` en de scheduler verdeelt. (3) **Updaten**: `compose up` maakt de container opnieuw aan, de dienst valt enkele seconden weg op elke VM; het Deployment doet een *rolling update* (lab 12), nieuwe Pod klaar vóór de oude weggaat, aangestuurd door de controller manager. Daar komt het zelfherstel bij **machinepanne** nog bij, dat `restart: always` niet dekt.

**Waarom.** Compose werkt op één machine, op aanvraag; het control plane werkt op alle machines, permanent, vanuit een gewenste toestand.

**Nuance.** Voor een interne applicatie zonder beschikbaarheidseis zijn de twee Compose-VM's een redelijk antwoord (lab 09). Het verschil betaal je in complexiteit.

**Voorbeeld.**

```
kubectl get deployment api -o jsonpath='{.spec.replicas} gewenst, {.status.readyReplicas}
klaar'
```

---

### Vraag 2 — `apply`, `apply`, `create`

**Antwoord.** `pod/api created`, dan `pod/api unchanged`, dan `Error from server (AlreadyExists): … pods "api" already exists`. **Declaratief**: het bestand beschrijft een toestand, geen handeling; `apply` vergelijkt de gewenste toestand met het bestaande object en werkt alleen op het verschil. **Idempotent**: hetzelfde bestand N keer toepassen geeft hetzelfde resultaat als één keer — wat `apply` veilig maakt in CI, in een script, of na een panne. `create` is imperatief: "maak aan", en faalt dus als het object bestaat.

**Waarom.** `apply` bewaart de laatst toegepaste configuratie (annotatie `kubectl.kubernetes.io/last-applied-configuration`) en berekent een driewegs-*patch*.

**Nuance.** Sommige velden van een Pod zijn **onveranderlijk** (de image, bijvoorbeeld): een `apply` die ze wijzigt, faalt; je moet verwijderen en opnieuw aanmaken. Deployments (lab 12) bestaan precies om de image te veranderen zonder die hinder.

**Voorbeeld.**

```
kubectl apply -f pod-api.yaml ; kubectl apply -f pod-api.yaml      # created, dan unchanged
kubectl get pod api -o jsonpath='{.metadata.annotations}' | cut -c1-80
```

---

### Vraag 3 — Een losse Pod, een Pod onder een Deployment

**Antwoord.** De laatste `get pods` toont **één enkele** Pod, `api-xxxxxxxxx-yyyyy`, enkele seconden geleden aangemaakt; `web` is verdwenen. `web` is een Pod **zonder eigenaar**: verwijderd, eist

niemand zijn terugkeer. De Pod van `api` hoort bij een **ReplicaSet**, zelf aangemaakt door het **Deployment**: de reconciliatielus van de ReplicaSet stelt `0 < 1` replica vast en maakt een nieuwe aan, met een ander achtervoegsel.

**Waarom.** Elk object kan een `ownerReference` dragen; een controller bewaakt de objecten die hij bezit en houdt het aantal op peil.

**Nuance.** De nieuwe Pod heeft een **nieuw IP** en een nieuwe naam: niets wat naar de oude verwees, mag vast ingeschreven zijn — het argument voor de Services van lab 13.

**Voorbeeld.**

```
kubectl get pod -l app=api -o jsonpath='{.items[0].metadata.ownerReferences[0].kind}'   # ReplicaSet
```

---

### Vraag 4 — `ImagePullBackOff` op een lokale image

**Antwoord.** De kubelet heeft **zijn** containerd gevraagd om `localhost/api:1.0` te halen; die containerd kent de image niet (ze zit in de opslag van Podman, een andere map, een andere gebruiker) en probeert een pull van een registry `localhost` die niet bestaat: `ErrImagePull`, dan `ImagePullBackOff` (nieuwe pogingen met toenemende tussenpozen). Oplossing: `podman save localhost/api:1.0 | sudo k3s ctr images import -`; de kubelet probeert opnieuw en vindt de image. Het veld dat ertoe doet: `imagePullPolicy: IfNotPresent` (of `Never`), anders zou `Always` zelfs een geïmporteerde image laten falen.

**Waarom.** Twee runtimes, twee opslagplaatsen: rootless Podman (`~/.local/share/containers`) en de containerd van k3s (`/var/lib/rancher/k3s`).

**Nuance.** `Always` is de standaard wanneer de tag `latest` is of ontbreekt; `IfNotPresent` voor elke andere tag. Het beleid expliciet schrijven vermijdt de verrassing.

**Voorbeeld.**

```
sudo k3s crictl images | grep api        # leeg vóór de import, één regel erna
kubectl describe pod api | grep -A2 'Failed to pull'
```

---

### Vraag 5 — De onzichtbare Pod

**Antwoord.** De Pod zit in een **andere namespace** dan die van de huidige context (`default`). Om hem te zien: `kubectl get pods -n lab11`, of `-A` (alle namespaces). Om `-n` niet meer te herhalen: `kubectl config set-context --current --namespace=lab11`, dat de standaardnamespace van de context in de kubeconfig wijzigt.

**Waarom.** Een namespace is een afscherming: elk `kubectl`-commando geldt voor één enkele, die van de optie `-n` of van de context.

**Nuance.** Sommige objecten zijn **niet** namespaced (nodes, namespaces, StorageClasses, PersistentVolumes): `kubectl api-resources` geeft het aan in de kolom `NAMESPACED`.

**Voorbeeld.**

```
kubectl config view --minify -o jsonpath='{.contexts[0].context.namespace}'   # lab11
```

---

### Vraag 6 — Eén Pod of twee

**Antwoord.** (a) nginx schrijft `proxy_pass http://localhost:8080/`: dezelfde network namespace, zoals de Podman-pod van **lab 07**. (b) nginx schrijft `proxy_pass http://api:8080/`, een naam die een **Service** resolvet: zoals het user-defined netwerk en Compose van **lab 09**. In productie: (b): elke dienst wordt **onafhankelijk** geschaald, bijgewerkt en herstart; met (a) zouden drie replica's van de frontend drie replica's van de API opleggen, en zou een update van de ene de andere herstarten.

**Waarom.** De Pod is de eenheid van plaatsing en replicatie: wat in dezelfde Pod zit, leeft en sterft samen.

**Nuance.** Meerdere containers dienen voor *sidecars* (logagent, TLS-proxy) en *init containers*: metgezellen van de hoofdcontainer, geen andere diensten.

**Voorbeeld.**

```
kubectl explain pod.spec.initContainers | head -5
```

---

### Vraag 7 — De kubeconfig van k3s

**Antwoord.** Drie elementen: het **cluster** (`server: https://127.0.0.1:6443` en het certificaat van de autoriteit), de **gebruiker** (een clientcertificaat en zijn sleutel, die de beheerder van het cluster is), en de **context** `default` die ze verbindt. In modus `644` leest elke gebruiker van de machine de sleutel en wordt hij beheerder: aanvaardbaar op je werkstation waar je alleen bent, onaanvaardbaar op een gedeelde server. Elders gekopieerd wijst het bestand naar `127.0.0.1`: je moet het adres vervangen door dat van de k3s-machine (en poort 6443 moet bereikbaar zijn) — het servercertificaat aanvaardt standaard het IP van de node.

**Waarom.** De kubeconfig is tegelijk het adresboek en de sleutelbos van `kubectl`.

**Nuance.** In het bedrijf bevat de kubeconfig een *token* of een OIDC-authenticatie met beperkte duur, geen eeuwig beheerderscertificaat.

**Voorbeeld.**

```
grep -E 'server:|client-certificate-data|name:' /etc/rancher/k3s/k3s.yaml | cut -c1-60
```

---

### Vraag 8 — `localhost:8080 refused`

**Antwoord.** `kubectl` vindt **geen enkele kubeconfig** en valt terug op zijn historische standaard, `http://localhost:8080`, waar niets luistert (de API server zit in HTTPS op 6443). Dat gebeurt wanneer je een apart geïnstalleerde `kubectl` gebruikt (`apt`, `snap`), die `/etc/rancher/k3s/k3s.yaml` niet kent. Correcties: `export KUBECONFIG=/etc/rancher/k3s/k3s.yaml` (in `~/.bashrc`), of `mkdir -p ~/.kube && cp /etc/rancher/k3s/k3s.yaml ~/.kube/config`.

**Waarom.** De `kubectl` die k3s levert, is een link naar de binary `k3s`, die weet waar zijn kubeconfig staat; een generieke `kubectl` weet dat niet.

**Nuance.** Het bestand naar `~/.kube/config` kopiëren maakt er **jouw** kopie van: als k3s zijn certificaten vernieuwt, is ze verouderd. De variabele `KUBECONFIG` volgt altijd het origineel.

**Voorbeeld.**

```
which kubectl ; ls -l $(which kubectl)       # /usr/local/bin/kubectl -> k3s
KUBECONFIG=/etc/rancher/k3s/k3s.yaml kubectl get nodes
```

---

### Vraag 9 — API server stilgevallen

**Antwoord.** De Pods **blijven draaien**: de kubelet heeft ze gestart en bewaakt ze lokaal; het verkeer dat erop aankomt, wordt bediend. Je kunt niets meer **wijzigen of bekijken**: `kubectl` faalt, geen nieuwe Pod wordt gepland, geen Deployment wordt gereconcilieerd. Een Pod die sterft, wordt door de kubelet **op dezelfde node** herstart (hij past de `restartPolicy` toe), maar is er een andere node nodig, dan beslist niemand. De API server is dus de deur om te **wijzigen en waar te nemen**; de echte motor is de kubelet, die in het geheugen houdt wat hij moet laten draaien.

**Waarom.** Kubernetes is zo ontworpen dat het dataplane het control plane overleeft: elke node is autonoom voor wat hij al heeft.

**Nuance.** Met k3s op één node betekent "de API server stopt" `systemctl stop k3s`, wat **ook** de kubelet en containerd stopt: de Pods sterven. De autonomie van de nodes zie je alleen op een echt cluster.

**Voorbeeld.**

```
kubectl cluster-info          # Kubernetes control plane is running at https://127.0.0.1:6443
kubectl get --raw /healthz    # ok
```

---

### Vraag 10 — De Events van een start

**Antwoord.** `Scheduled` — de **scheduler** koos de node `wsl-ubuntu` (hier de enige). `Pulling` / `Pulled` — de **kubelet** vroeg containerd de image te halen (3,75 s). `Created` / `Started` — de kubelet maakte de container aan en startte hem via containerd. Tegenover `podman run` van lab 03: pull, aanmaak en start zijn dezelfde stappen, gedaan door `podman` en `crun`; het cluster voegt de **planning** toe (een machine kiezen), het **registreren** van elke stap als Event in de API server, en het **voortdurende toezicht**. Met één node heeft de scheduler geen keuze: hij "werkt" zonder te beslissen.

**Waarom.** Elk onderdeel publiceert Events op het object dat het behandelt; `describe` verzamelt ze in volgorde.

**Nuance.** Events worden standaard één uur bewaard: voor een ouder incident heb je de logs van de applicatie of een externe verzameling nodig.

**Voorbeeld.**

```
kubectl get events --sort-by=.lastTimestamp | tail -5
```

---

### Vraag 11 — `Pending` zonder IP

**Antwoord.** Ofwel **past geen enkele node** (onvoldoende resources, node `NotReady`, plaatsingsbeperking), ofwel **blokkeert de image of het volume** vóór de toewijzing — maar zonder enig IP is het bijna altijd de scheduler. `kubectl describe pod` hakt de knoop door: `Warning FailedScheduling … 0/1 nodes are available: 1 Insufficient memory` aan de ene kant; `Scheduled` gevolgd door een `Failed to pull image` of een `FailedMount` aan de andere (met al een toegewezen IP).

**Waarom.** Het IP wordt door de CNI toegekend op het moment dat de kubelet de Pod op een node aanmaakt; geen node, geen IP.

**Nuance.** Een Pod die `resources.requests.memory: 8Gi` vraagt op een WSL met 4 GB blijft voor altijd `Pending`, zonder foutmelding elders dan in zijn Events.

**Voorbeeld.**

```
kubectl describe pod api | grep -A3 '^Events' ; kubectl get nodes ; kubectl describe node |
grep -A5 'Allocated resources'
```

---

### Vraag 12 — Van k3s naar EKS

**Antwoord.** Aan te vragen of in te stellen: (1) een **Ingress controller** (Traefik zit in k3s; op EKS installeer je ingress-nginx of gebruik je de ALB); (2) een standaard-**StorageClass** (`local-path` bij k3s; op EKS, EBS via een CSI-driver); (3) Services van het type `LoadBalancer` (ServiceLB bij k3s; een betalende cloud-load-balancer op EKS); (4) de **toegang tot de images**: geen `k3s ctr images import` meer, de images komen uit een registry (ECR) met pull-rechten. Wat niet verandert: de **manifesten** (Pod, Deployment, Service, ConfigMap…) en de `kubectl`-commando's.

**Waarom.** k3s verpakt keuzes die beheerde clusters open laten, omdat ze afhangen van de infrastructuur van de klant.

**Nuance.** Ook op EKS heeft het platformteam die keuzes vaak al gemaakt: de ontwikkelaar **ontdekt** ze (`kubectl get storageclass`, `kubectl get ingressclass`) eerder dan ze te installeren.

**Voorbeeld.**

```
kubectl get storageclass ; kubectl get ingressclass ; kubectl get svc -n kube-system traefik
```

---

### Vraag 13 — `hostPath` en `hostPort`

**Antwoord.** `hostPath` mount een bestand **van de machine die de Pod uitvoert**: op een cluster kan de Pod op gelijk welke node terechtkomen, waar dat bestand niet bestaat — en een Pod die de schijf van de nodes leest, is een beveiligingslek (een `hostPath` op `/etc` leest de configuratie van de host). `hostPort` reserveert een poort **van de node**: twee replica's op dezelfde node botsen, en de client moet het adres van de node kennen. Vervangingen: de nginx-configuratie wordt een **ConfigMap** als volume gemount (lab 14); de blootstelling wordt een **Service** en dan een **Ingress** (lab 13), die van geen enkele node afhangen.

**Waarom.** Een clustermanifest moet **onafhankelijk van de machine** zijn: alles wat de Pod nodig heeft, is een object van de API, geen lokaal pad.

**Nuance.** `hostPath` en `hostPort` hebben legitieme toepassingen voor **node-agents** (logverzamelaar, monitoring) die als DaemonSet uitgerold worden, één per machine.

**Voorbeeld.**

```
grep -n -E 'hostPath|hostPort' ~/labo-docker/11/demo-pod.yaml
```